

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра програмної інженерії

КУРСОВА РОБОТА

Пояснювальна записка

з дисципліни "Об'єктно-орієнтоване програмування"

ДОВІДНИК АБІТУРІЄНТА

Виконав:

здобувач _1_ року навчання, групи ПЗП-25-3

_____ Єлизавета Головка

Керівник: _____ Проф. Володимир БОНДАРЄВ

Комісія:

_____ Проф. Володимир БОНДАРЄВ

_____ Ст. викл. Юлія ЧЕРЕПАНОВА

_____ Ст. викл. Віталій ЛЯПОТА

Харків 2026

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ

Кафедра *програмної інженерії*

Рівень вищої освіти *перший (бакалаврський)*

Дисципліна *Об'єктно-орієнтоване програмування*

Спеціальність *121 Інженерія програмного забезпечення*

Освітня програма: *Програмна інженерія*

Курс 1.

Група ПЗП-25-3.

Семестр 2.

ЗАВДАННЯ

на курсову роботу здобувачеві

Головко Єлизаветі Андріївні

1 Тема роботи

Довідник абітурієнта

2 Термін подання здобувачем роботи до захисту **“30” травня 2026 р.**

3 Вихідні дані до роботи

Програма зберігає базу вузів: найменування, адреса, перелік спеціальностей, конкурс минулого року за кожною спеціальністю (денною, вечірньою, заочною формами), розмір оплати при договірному навчанні. Вибір за різними критеріями: все щодо обраного вузу; все щодо обраної спеціальності, пошук мінімального конкурсу з даної спеціальності та інше.

4 Перелік питань, що потрібно опрацювати в роботі

Вступ, опис вимог, проєктування програми, структура класів, спосіб зберігання даних, інструкція користувача, висновки.

КАЛЕНДАРНИЙ ПЛАН

№	Назва етапів роботи	Термін виконання
1	Видача теми, узгодження і затвердження теми (1 тижд.)	25.03.2026 – 01.04.2026 р.
2	Формулювання мети роботи (1 тижд.)	01.04.2026 – 08.04.2026 р.
3	Опис вимог до програми (1 тижд.)	08.04.2026 – 15.04.2026 р.
4	Проектування програми (1 тижд.)	15.04.2026 – 22.04.2026 р.
5	Кодування (2 тижд.)	22.04.2026 – 06.05.2026 р.
6	Підготовка роботи до захисту (1 тижд.)	13.05.2026 – 20.05.2026 р.
7	Захист	20.05.2026 – 30.05.2026 р.

Здобувач _____  _____ Головка Єлизавета Андріївна

Керівник _____ Проф. Володимир БОНДАРЄВ

«28» травня 2026 р.

РЕФЕРАТ

Пояснювальна записка до курсової роботи: 33 с., 10 рис., 7 джерел.

ДОВІДНИК АБІТУРІЄНТА, ЗАКЛАД ВИЩОЇ ОСВІТИ, СПЕЦІАЛЬНІСТЬ, КОНКУРСНИЙ БАЛ, ВАРТІСТЬ НАВЧАННЯ, WINDOWS FORMS, C#, ООП, .NET.

Метою роботи є розробка програми «Довідник абітурієнта», яка призначена для автоматизації процесів обліку, систематизації та швидкого пошуку інформації про заклади вищої освіти та їхні освітні програми.

У результаті розроблено програмний продукт, що дозволяє зберігати інформацію про університети із зазначенням назви, адреси та міста. Для кожного університету ведеться перелік спеціальностей, де фіксуються прохідні бали минулого року на денну, вечірню та заочну форми навчання, а також розмір оплати при договірному навчанні. Також реалізовано можливість додавати, редагувати та видаляти дані, здійснювати багатофакторний пошук за різними критеріями (включно з пошуком мінімального конкурсу з даної спеціальності), а також автоматичне збереження та завантаження даних із локального файлу.

В процесі розробки використано середовище розробки Microsoft Visual Studio 2022, фреймворк Windows Forms, платформу .NET 8.0 та мову програмування C#. Дані програми зберігаються у локальному файлі.

ЗМІСТ

ВСТУП.....	6
1 ОПИС ВИМОГ	7
1.1 Сценарії використання	7
1.2 Функції програми	12
2 ПРОЄКТУВАННЯ ПРОГРАМИ	22
2.1 Архітектура програми	22
2.2 UML-діаграма варіантів використання	23
2.3 UML-діаграма класів.....	25
2.4 Структура класів.....	27
2.5 Спосіб зберігання даних	28
2.6 Інструкція зі встановлення програми.....	29
ВИСНОВКИ	31
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ	33

ВСТУП

Сучасний етап розвитку суспільства характеризується стрімким зростанням обсягів інформації в усіх сферах людської діяльності. Сфера вищої освіти не є винятком: щороку тисячі абітурієнтів стикаються з проблемою вибору майбутньої професії та закладу вищої освіти (ЗВО) для продовження навчання. Ухвалення такого важливого рішення вимагає ретельного аналізу величезних масивів даних: переліку доступних спеціальностей, прохідних балів минулих років на різні форми навчання, вартості контракту, а також географічного розташування навчальних закладів.

Метою курсової роботи є розробка спеціалізованого програмного забезпечення «Довідник абітурієнта» з використанням принципів об'єктно-орієнтованого програмування. Програма призначена для автоматизації процесів збору, збереження, систематизації та зручного пошуку комплексної інформації про заклади вищої освіти. Головна задача проєкту – надати користувачу єдиний, надійний та зручний інструмент для оптимізації процесу вибору місця подальшого навчання на основі багатofакторного аналізу вхідних даних.

До впровадження розробленої програми типова діяльність абітурієнта або його представників зводилася до виснажливого ручного моніторингу інформації. Користувачу доводилося самотійно відвідувати десятки офіційних вебсайтів різних університетів, шукати актуальні прайс-листи на навчання та вручну виписувати прохідні бали минулих років. Такий децентралізований підхід не лише потребує значних витрат часу та зусиль, але й несе у собі високий ризик людської помилки. Через інформаційне перенавантаження користувач може легко пропустити заклад із меншим конкурсним балом або не помітити більш вигідні фінансові умови навчання у сусідньому регіоні.

Після отримання та розгортання розробленої програми діяльність користувача кардинально зміниться та перейде на якісно новий рівень. Замість багатогодинного пошуку розрізнених даних в мережі Інтернет, користувач працюватиме з консолідованою, чітко структурованою інформацією в єдиному графічному вікні. Це дозволить повністю делегувати рутинну роботу з фільтрації алгоритмам програми та зосередитися виключно на аналітиці й виборі найкращого для себе варіанту.

1 ОПИС ВИМОГ

1.1 Сценарії використання

У цьому підрозділі наведено основні сценарії використання програми. Кожен сценарій описує дії користувача під час роботи з певною функцією програми, а також реакцію системи на некоректні дії. Основними сценаріями використання програми «Довідник абітурієнта» є:

- 1) перегляд загального довідника університетів;
- 2) пошук спеціальності за назвою;
- 3) фільтрація спеціальностей за формою навчання;
- 4) фільтрація за максимальним фінансовим лімітом;
- 5) пошук за географічним критерієм (містом);
- 6) додавання нового ЗВО;
- 7) редагування даних існуючого ЗВО;
- 8) оновлення вартості навчання;
- 9) видалення закладу вищої освіти;
- 10) повне очищення бази даних;
- 11) видалення запису з результатів пошуку;
- 12) збереження та завантаження даних.

1.1.1 Сценарій «Перегляд загального довідника університетів»

Передумова: користувач запустив програму, відкрито вкладку «Довідник ВНЗ».

Основний сценарій:

- 1) програма автоматично завантажує базу даних з локального файлу;
- 2) у лівій таблиці відображається перелік усіх збережених університетів;
- 3) користувач клікає лівою кнопкою миші на обраний рядок з університетом;
- 4) програма обробляє вибір та миттєво заповнює праву таблицю даними;
- 5) у правій таблиці відображаються всі спеціальності обраного закладу, прохідні бали на денну, вечірню та заочну форми навчання, а також вартість контракту.

1.1.2 Сценарій «Пошук спеціальності за назвою»

Передумова: користувач відкрив вкладку «Пошук».

Основний сценарій:

- 1) користувач вводить повну назву спеціальності або її фрагмент у поле «Назва спеціальності»;
- 2) користувач натискає кнопку «Знайти»;
- 3) програма ігнорує реєстр символів та шукає введений фрагмент серед усіх спеціальностей бази;
- 4) алгоритм вираховує найменший прохідний бал для кожної знайденої спеціальності та сортує список за зростанням цього бала;
- 5) відсортований список виводиться у вікно результатів, оновлюється лічильник знайдених записів.

Додатковий сценарій:

- 1) користувач вводить фрагмент назви, якого немає в базі;
- 2) програма аналізує дані і не знаходить жодного збігу;
- 3) у списку результатів виводиться повідомлення «На жаль, за вашим запитом нічого не знайдено», лічильник показує «0».

1.1.3 Сценарій «Фільтрація спеціальностей за формою навчання»

Передумова: користувач відкрив вкладку «Пошук».

Основний сценарій:

- 1) користувач вводить у поле «Форма навчання» одне зі значень: «денна», «вечірня» або «заочна»;
- 2) користувач натискає кнопку «Знайти»;
- 3) програма перевіряє наявність прохідного бала для вказаної форми (бал має бути більшим за нуль);
- 4) у результатах пошуку відображаються лише ті спеціальності, які здійснюють набір на обрану форму навчання.

Додатковий сценарій:

- 1) користувач залишає поле «Форма навчання» порожнім і натискає «Знайти»;
- 2) програма ігнорує цей критерій і виводить спеціальності з будь-якими формами навчання.

1.1.4 Сценарій «Фільтрація за максимальним фінансовим лімітом»

Передумова: користувач має обмежений бюджет на навчання за контрактом.

Основний сценарій:

- 1) користувач вводить ціле число у поле «Макс. вартість (грн)»;

- 2) користувач натискає «Знайти»;
- 3) програма порівнює введене число з вартістю кожної спеціальності;
- 4) спеціальності, вартість яких перевищує вказаний ліміт, відсіюються з результатів.

Додатковий сценарій:

- 1) користувач вводить літери або спеціальні символи замість цифр у поле вартості;
- 2) при натисканні «Знайти» програма не може розпізнати число;
- 3) фільтр за вартістю ігнорується, програма продовжує пошук за іншими критеріями без збоїв у роботі.

1.1.5 Сценарій «Пошук за географічним критерієм - містом»

Передумова: користувач бажає навчатися у конкретному регіоні.

Основний сценарій:

- 1) користувач вводить назву міста в текстове поле «Місто»;
- 2) користувач натискає «Знайти»;
- 3) алгоритм перевіряє властивість «Місто» для кожного університету;
- 4) заклади з інших міст відкидаються, на екран виводяться спеціальності лише з потрібного регіону.

1.1.6 Сценарій «Додавання нового ЗВО»

Передумова: користувач відкрив вкладку «Довідник ВНЗ».

Основний сценарій:

- 1) користувач натискає кнопку «Додати ВНЗ»;
- 2) у модальному вікні користувач заповнює текстові поля «Найменування», «Адреса» та «Місто»;
- 3) користувач натискає кнопку «ОК»;
- 4) програма перевіряє дані, створює новий запис, зберігає його у файл бази даних та оновлює таблицю на екрані.

Додатковий сценарій:

- 1) користувач залишає обов'язкове поле «Найменування» порожнім і натискає «ОК»;
- 2) програма блокує транзакцію збереження;

- 3) на екран виводиться повідомлення про помилку: «Поле 'Найменування' є обов'язковим для заповнення!»;
- 4) вікно не закривається, даючи змогу виправити помилку.

1.1.7 Сценарій «Редагування даних існуючого ЗВО»

Передумова: змінилися фактичні дані навчального закладу.

Основний сценарій:

- 1) користувач виділяє заклад у лівій таблиці та натискає «Редагувати»;
- 2) програма відкриває вікно і переносить туди поточні дані ЗВО;
- 3) користувач вносить виправлення та натискає «ОК»;
- 4) об'єкт оновлюється, зміни автоматично зберігаються у файл, таблиця оновлюється.

Додатковий сценарій:

- 1) користувач натискає «Редагувати», не виділивши жодного університету в таблиці;
- 2) програма перехоплює дію і виводить попередження: «Будь ласка, спочатку виберіть університет!»;
- 3) вікно редагування не відкривається.

1.1.8 Сценарій «Оновлення вартості навчання»

Передумова: користувач знайшов потрібну спеціальність на вкладці «Пошук».

Основний сценарій:

- 1) користувач виділяє рядок зі спеціальністю у результатах пошуку та натискає «Оновити вартість»;
- 2) у модальному вікні користувач вводить нове значення ціни і натискає «ОК»;
- 3) програма замінює стару вартість контракту новою, ініціює збереження у файл та автоматично оновлює список результатів пошуку на екрані.

Додатковий сценарій:

- 1) у поле нової вартості користувач вводить текст замість цифр і натискає «ОК»;
- 2) програма фіксує неможливість конвертації формату;
- 3) виводиться попередження «Будь ласка, введіть коректне число для вартості!», збереження скасовується.

1.1.9 Сценарій «Видалення закладу вищої освіти»

Передумова: університет припинив свою діяльність.

Основний сценарій:

- 1) користувач виділяє ЗВО у лівій таблиці та натискає «Видалити»;
- 2) програма виводить попередження: «Ви дійсно хочете назавжди видалити цей університет з бази?»;
- 3) користувач натискає «Так»;
- 4) заклад та всі його спеціальності видаляються з пам'яті, файл перезаписується, таблиці очищуються.

Додатковий сценарій:

- 1) після появи вікна підтвердження користувач усвідомлює помилку і натискає «Ні»;
- 2) операція негайно переривається, всі дані залишаються недоторканими.

1.1.10 Сценарій «Повне очищення бази даних»

Передумова: користувачу необхідно повністю стерти всі записи.

Основний сценарій:

- 1) користувач натискає кнопку повного очищення бази;
- 2) з'являється критичне попередження про незворотність дії;
- 3) користувач підтверджує намір, натискаючи «Так»;
- 4) програма очищує пам'ять, створює порожній файл збереження та повністю очищує візуальні таблиці інтерфейсу.

Додатковий сценарій:

- 1) користувач випадково натиснув кнопку очищення;
- 2) у вікні попередження користувач натискає «Ні»;
- 3) очищення скасовується.

1.1.11 Сценарій «Видалення запису з результатів пошуку»

Передумова: користувач хоче візуально відсіяти нецікавий результат пошуку.

Основний сценарій:

- 1) користувач виділяє рядок у списку знайдених спеціальностей та натискає кнопку «Видалити запис»;
- 2) з'являється запит на підтвердження;
- 3) після натискання «Так» програма прибирає рядок з екрана та перераховує лічильник результатів.

Додатковий сценарій:

- 1) користувач натискає кнопку, не виділивши жодного рядка;
- 2) програма виводить попередження «Виберіть запис для видалення!».

1.1.12 Сценарій «Збереження та завантаження даних»

Передумова: програма запускається або користувач змінює дані.

Основний сценарій завантаження:

- 1) при старті програма зчитує локальний файл збереження;
- 2) програма відновлює всі об'єкти університетів та спеціальностей і виводить їх у таблиці.

Основний сценарій збереження:

- 1) після будь-якої зміни даних (додавання, редагування, видалення) програма миттєво і у фоновому режимі ініціює збереження оновленої колекції у файл.

Додатковий сценарій :

- 1) користувач запускає програму вперше, файл збереження відсутній або пошкоджений;
- 2) програма перехоплює помилку читання;
- 3) програма автоматично генерує та зберігає нову еталонну базу з провідних університетів України, що запобігає збою роботи додатка.

1.2 Функції програми

Функції програми є програмною реалізацією сценаріїв використання, що були описані у попередньому підрозділі. Програма «Довідник абітурієнта» має сучасний графічний інтерфейс користувача (GUI), логічно поділений на дві основні вкладки: «Довідник ВНЗ» та «Пошук». Такий архітектурний поділ

дозволяє чітко розмежувати процеси адміністрування бази даних та процеси аналітичного пошуку інформації.

Основними функціями програми є:

1. ведення та відображення загального списку університетів і спеціальностей;
2. багатofакторний пошук та фільтрація даних;
3. сортування результатів за мінімальним конкурсним балом;
4. додавання, редагування та видалення записів про ЗВО;
5. актуалізація (оновлення) вартості контрактного навчання;
6. автоматичне збереження, завантаження та генерація еталонних даних.

1.2.1 Функція «Ведення загального списку університетів і спеціальностей»

Функція призначена для зручного візуального перегляду всієї наявної в базі інформації. Для роботи з цією функцією використовується перша вкладка головного вікна програми – «Довідник ВНЗ». Загальний вигляд цієї вкладки наведено на рисунку 1.1.

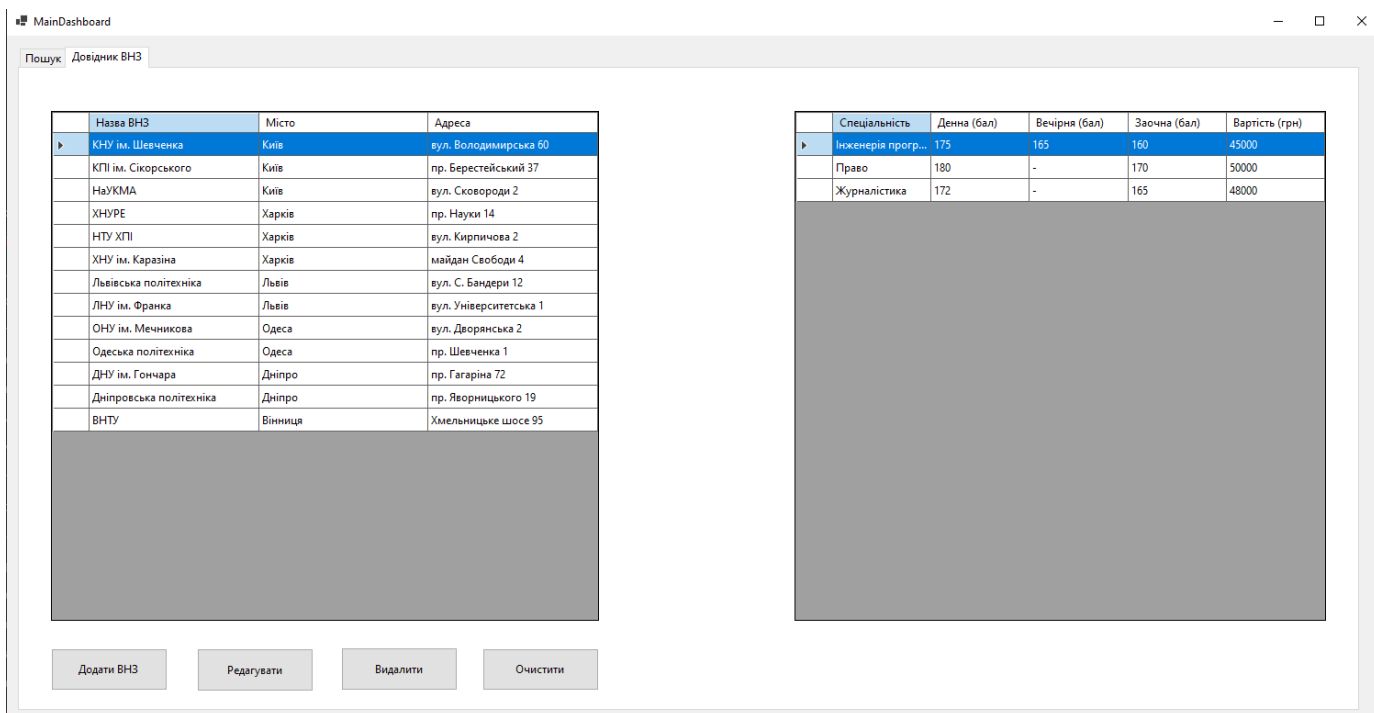


Рисунок 1.1 – Вкладка «Довідник ВНЗ» головного вікна програм

Робоча область вкладки візуально поділена на дві взаємопов'язані таблиці (компоненти DataGridView). У лівій частині розташована таблиця загального переліку закладів вищої освіти. Вона містить наступні інформаційні колонки:

- 1) «Назва ВНЗ» – відображає офіційне найменування університету;
- 2) «Місто» – вказує на географічне розташування закладу;
- 3) «Адреса» – містить точні координати головного корпусу або приймальної комісії.

Таблиця налаштована у режимі читання ReadOnly, що унеможливлює випадкову зміну даних безпосередньо у комірках. Виділення відбувається цілими рядками FullRowSelect, що підвищує зручність навігації.

У правій частині вкладки розташована таблиця спеціальностей, яка динамічно оновлюється. При виділенні будь-якого університету у лівій таблиці, програма автоматично підвантажує і виводить у праву таблицю перелік освітніх програм обраного ЗВО. Права таблиця містить п'ять колонок:

- 1) «Спеціальність» – назва напрямку підготовки;
- 2) «Денна (бал)» – прохідний бал минулого року на денну форму навчання;
- 3) «Вечірня (бал)» – прохідний бал на вечірню форму;
- 4) «Заочна (бал)» – прохідний бал на заочну форму;
- 5) «Вартість (грн)» – вартість одного року навчання за контрактом. Якщо набір на певну форму навчання не проводиться, у відповідній комірці виводиться прочерк «-».

Під таблицями розташована панель інструментів для адміністрування бази. Вона містить кнопки: «Додати ВНЗ», «Редагувати», «Видалити» та кнопку глобального очищення бази даних «Очистити».

1.2.2 Функція «Багатофакторний пошук та сортування»

Функція пошуку є ключовим аналітичним інструментом програми, що дозволяє абітурієнту швидко знайти оптимальне місце для вступу. Цей функціонал реалізовано на вкладці «Пошук». Загальний вигляд вкладки наведено на рисунку 1.2.

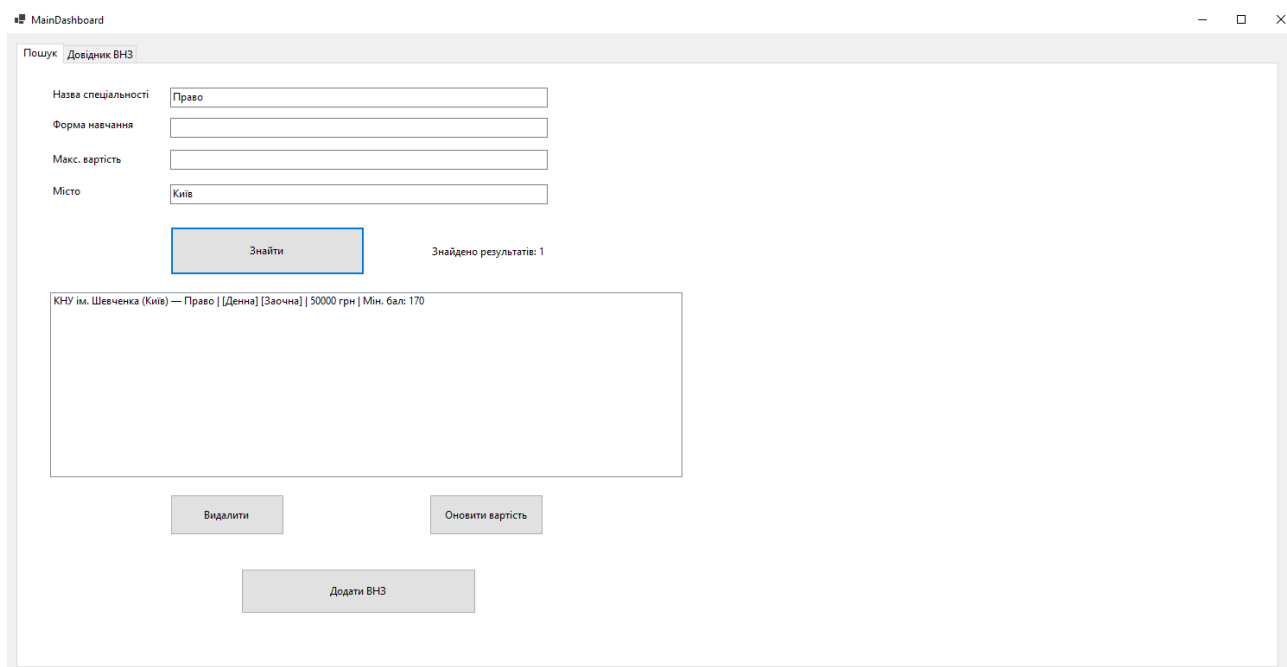


Рисунок 1.2 – Вкладка «Пошук» та результати фільтрації

У верхній частині вкладки розташована панель фільтрів, яка містить чотири незалежні текстові поля для введення критеріїв пошуку:

- 1) «Назва спеціальності» – поле для введення ключового слова або повної назви освітньої програми;
- 2) «Форма навчання» – поле для фільтрації за типом здобуття освіти (приймає значення «денна», «вечірня» або «заочна»);
- 3) «Макс. вартість (грн)» – поле для встановлення фінансового ліміту на контрактне навчання;
- 4) «Місто» – поле для звуження пошуку до конкретного населеного пункту.

Користувач може заповнювати як усі поля одночасно, для жорсткої фільтрації, так і залишати частину полів порожніми - у такому разі відповідний критерій просто ігнорується алгоритмом. Пошук є нечутливим до регістру символів.

Після натискання кнопки «Знайти», програма обробляє масив даних і виводить результати у велике текстове поле ListBox у нижній частині вікна. Кожен знайдений запис форматується у єдиний інформативний рядок, що містить назву ЗВО, місто, спеціальність, доступні форми навчання та вартість.

Особливістю даної функції є інтегрований алгоритм визначення мінімального конкурсу. Програма автоматично аналізує прохідні бали для кожної знайденої спеціальності, обирає найменший з них і виводить його в кінці рядка,

наприклад, «Мін. бал: 145». Увесь фінальний список результатів автоматично сортується за цим мінімальним балом у порядку зростання.

У нижній частині вкладки також розташований динамічний лічильник «Знайдено результатів: X», який інформує користувача про загальну кількість збігів. Крім того, на вкладці присутні кнопки «Оновити вартість» для зміни ціни вибраного запису та «Видалити запис» для візуального приховування рядка з екрана.

1.2.3 Функція «Додавання нового закладу вищої освіти»

Функція призначена для розширення бази даних шляхом внесення інформації про нові університети. Для реалізації цієї функції розроблено окреме вікно, яке відкривається у модальному режимі. Це означає, що користувач не може повернутися до головного вікна, поки не завершить або не скасує процес додавання. Загальний вигляд вікна додавання наведено на рисунку 1.3.

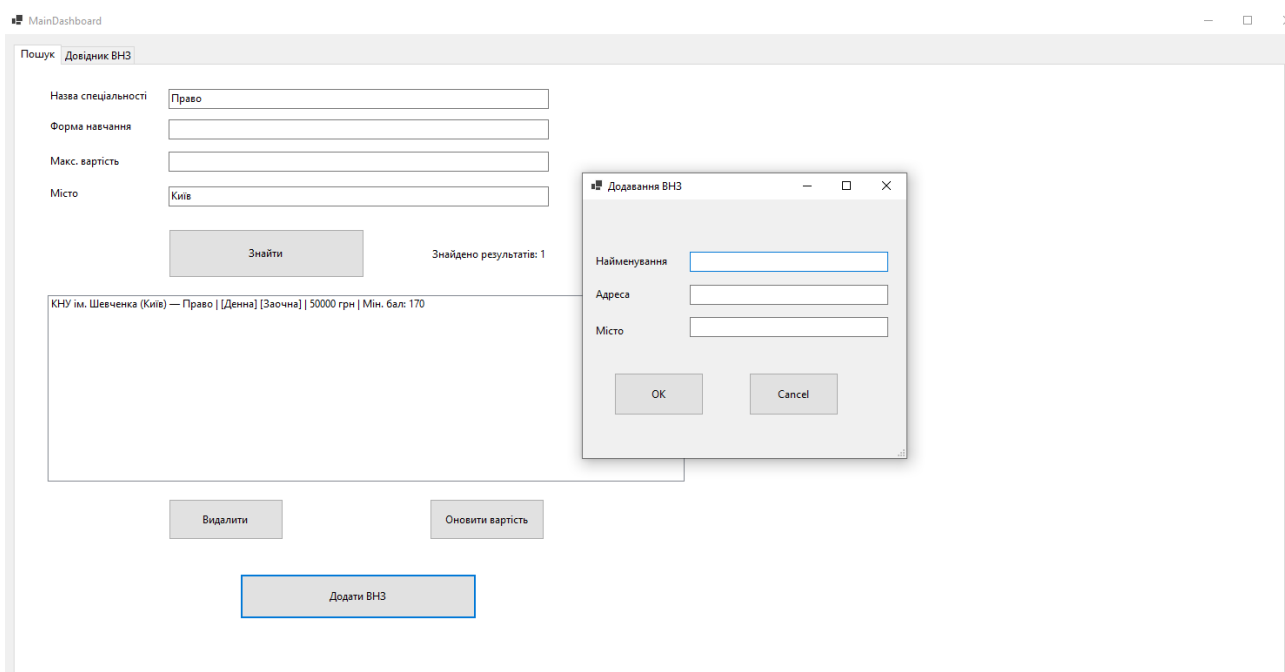


Рисунок 1.3 – Вікно додавання нового ВНЗ

Інтерфейс вікна містить три текстові поля TextBox:

- 1) «Найменування» – поле для введення офіційної назви університету. Це поле є обов'язковим для заповнення.
- 2) «Адреса» – поле для введення вулиці та номера будинку (необов'язкове).
- 3) «Місто» – поле для вказання населеного пункту (необов'язкове).

Внизу вікна розташовані дві кнопки: «ОК» («Додати») та Cancel («Скасувати»). При натисканні кнопки додавання програма обов'язково перевіряє вміст поля «Найменування». Якщо воно порожнє, процес зупиняється і на екран виводиться інформаційне повідомлення про помилку. Якщо дані коректні, вікно закривається, новий об'єкт ЗВО створюється в пам'яті і відразу відображається у головній таблиці.

1.2.4 Функція «Редагування даних існуючого ЗВО»

Функція редагування дозволяє актуалізувати контактну інформацію університетів. Програмно ця функція використовує те саме модальне вікно, що і для додавання, але з динамічною зміною заголовка на «Редагування ВНЗ». Загальний вигляд вікна під час процесу редагування наведено на рисунку 1.4.

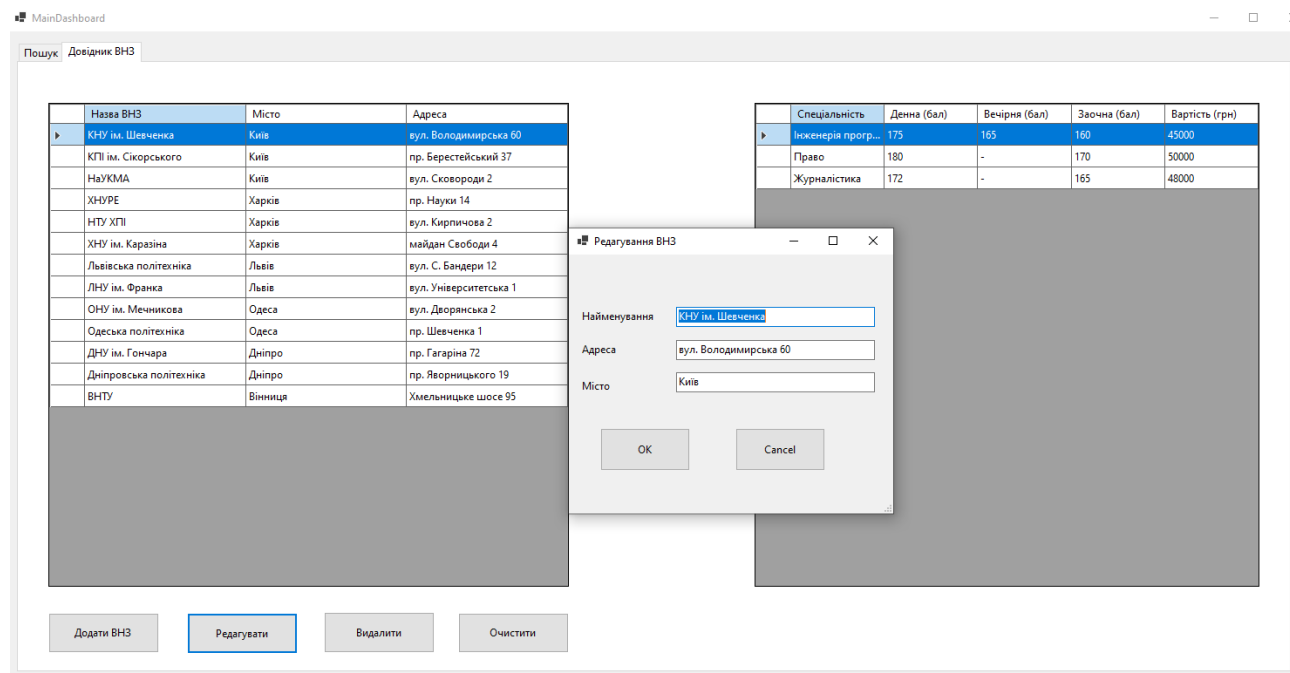


Рисунок 1.4 – Вікно редагування даних ЗВО

Головною особливістю цієї функції є механізм попереднього заповнення: при натисканні кнопки «Редагувати» на головній формі, програма зчитує властивості виділеного ЗВО і автоматично вставляє їх у текстові поля вікна редагування. Це звільняє користувача від необхідності вводити всю інформацію заново і дозволяє точково виправити лише помилку в адресі чи назві. Як і при додаванні, змінені дані проходять перевірку на порожнечу перед збереженням.

1.2.5 Функція «Оновлення вартості навчання»

Оскільки цінова політика ЗВО змінюється щороку, у програмі передбачено функцію швидкого оновлення вартості контракту для конкретної спеціальності. Ця функція доступна на вкладці «Пошук». Загальний вигляд вікна оновлення наведено на рисунку 1.5.

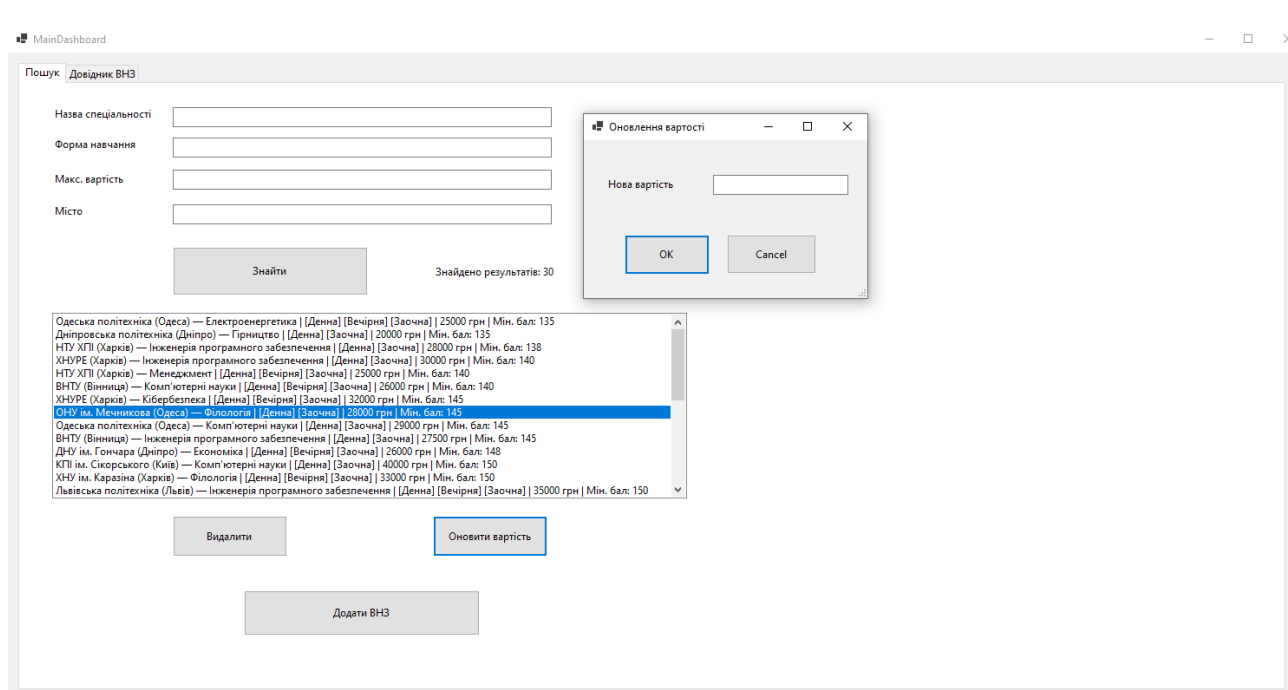


Рисунок 1.5 – Вікно оновлення вартості навчання

Інтерфейс представлений модальним вікном, яке містить єдине поле вводу «Нова вартість». Оскільки вартість має бути математичним значенням для коректної роботи фільтра максимальної ціни, програма здійснює сувору валідацію введених даних. Для перевірки використовується метод `decimal.TryParse`. Якщо користувач введе літери або залишить поле порожнім, програма виведе попередження: «Будь ласка, введіть коректне число для вартості!».

У разі успішного введення числа програма замінює значення в об'єкті спеціальності, зберігає базу і програмно імітує натискання кнопки пошуку, щоб користувач миттєво побачив оновлену ціну в результатах.

1.2.6 Функція «Видалення закладу вищої освіти»

Ця функція призначена для точкового видалення неактуальних університетів із загального реєстру. Кнопка «Видалити» розташована на першій вкладці «Довідник ВНЗ». Оскільки ця операція є деструктивною і видаляє не лише сам ЗВО, а й усі пов'язані з ним спеціальності, програма реалізує механізм захисту від випадкових натискань. Перед видаленням з'являється системне вікно

підтвердження MessageBox з іконкою попередження. Приклад цього вікна наведено на рисунку 1.6.

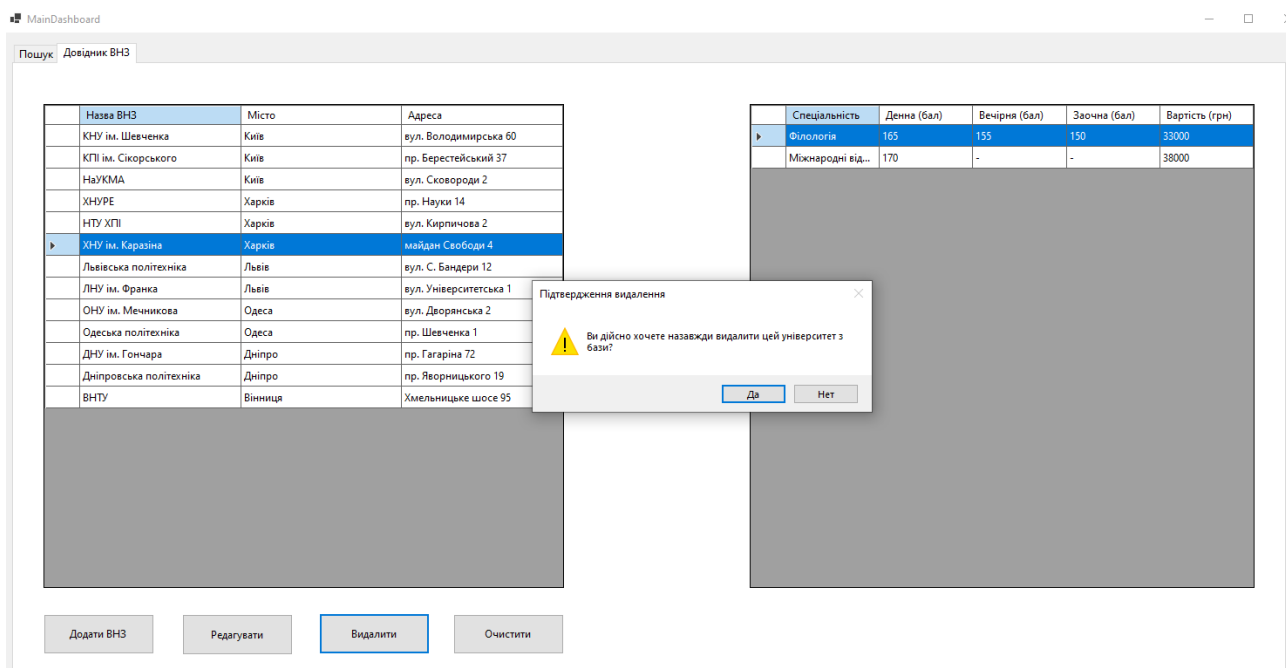


Рисунок 1.6 – Попередження при спробі видалення ЗВО

Лише після явного натискання кнопки «Так» програма виконує транзакцію видалення з оперативної пам'яті та перезаписує файл збереження.

1.2.7 Функція «Повне очищення бази даних»

Функція повного очищення використовується у випадках, коли користувачу необхідно стерти всі поточні масиви даних і повернути програму до нульового, тобто стартового стану. Для цього на вкладці «Довідник ВНЗ» передбачена кнопка «Очистити». Враховуючи критичність цієї дії, програма виводить вікно з іконкою зупинки Stop та червоним маркером небезпечної операції. Загальний вигляд цього попередження наведено на рисунку 1.7.

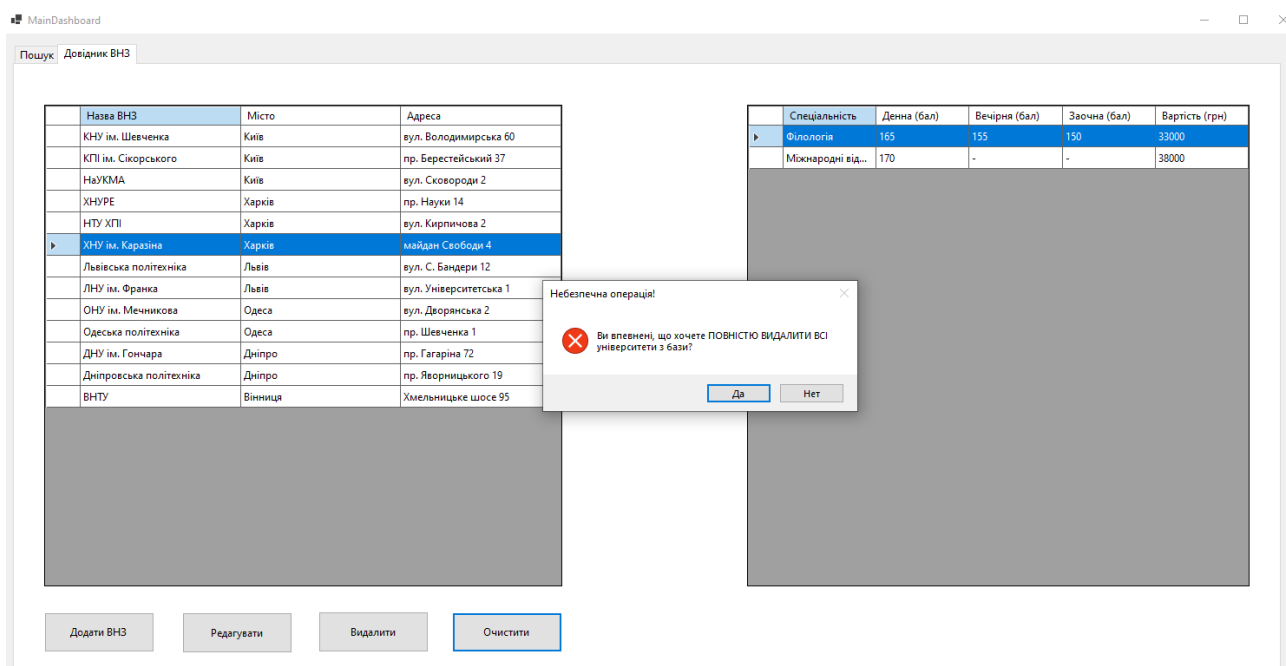


Рисунок 1.7 – Системне попередження при спробі повного очищення бази

Якщо користувач підтверджує намір, програма очищує всі колекції, створює порожній локальний файл та оновлює таблиці інтерфейсу, прибираючи всі візуальні дані.

1.2.8 Функція «Видалення запису з результатів пошуку»

Операція «Видалення запису з результатів пошуку» не є деструктивною для бази даних. Вона реалізована на вкладці «Пошук» і призначена виключно для візуального відсіювання результатів. Користувач може виділити нецікаву спеціальність у списку знайдених і натиснути кнопку «Видалити». Вікно підтвердження цієї дії показано на рисунку 1.8.

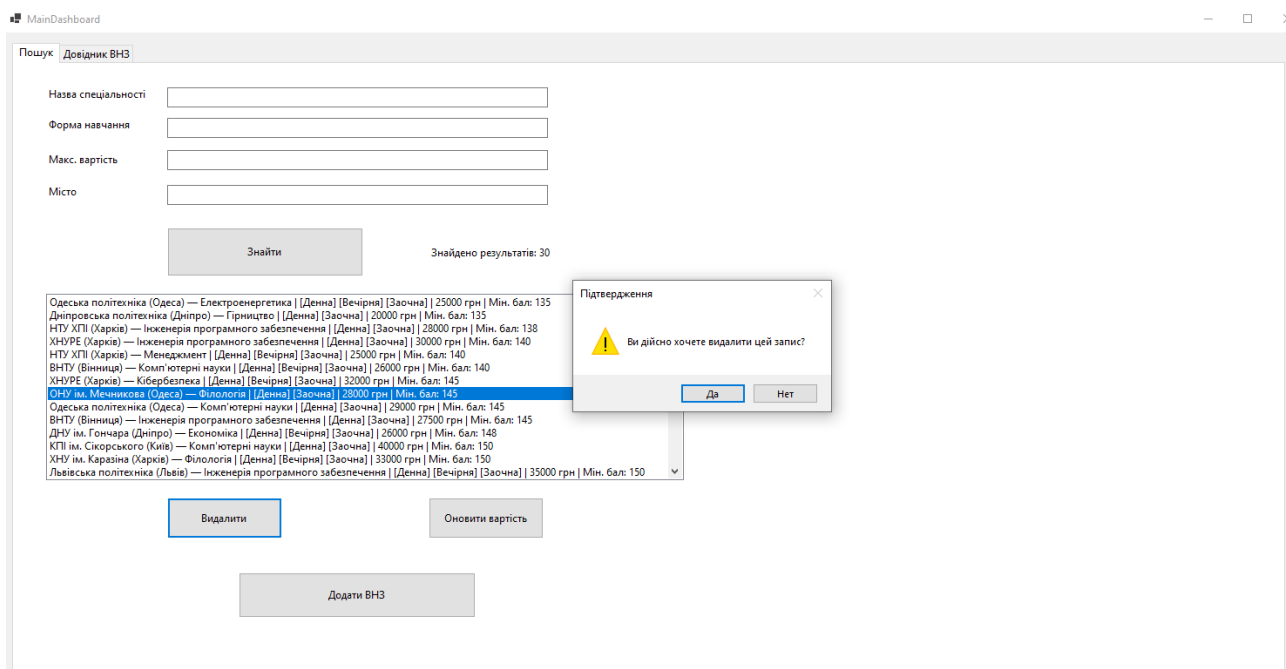


Рисунок 1.8 – Видалення рядка з візуальних результатів пошуку

Після підтвердження програма просто приховує обраний рядок з екрана для зручності подальшого аналізу та перераховує лічильник результатів, не пошкоджуючи при цьому інформацію у самому довіднику.

1.2.9 Функція «Збереження, завантаження та автоматична генерація»

Для забезпечення постійності даних між сеансами роботи у програмі реалізовано клас-менеджер AppManager. Дані зберігаються у текстовому форматі у локальному файлі безпосередньо в директорії програми. Особливістю функції є повна автоматизація: користувачу не потрібно натискати додаткові кнопки збереження. Програма ініціює збереження у фоновому режимі після кожної успішної транзакції (додавання, редагування, видалення, оновлення ціни).

При запуску програми відбувається спроба зчитати файл за допомогою конструкції try-catch. Якщо файл не знайдено, пошкоджено або в ньому міститься замало записів, спрацьовує підсистема генерації еталонних даних. Вона автоматично заповнює колекцію 13-ма провідними університетами України з детальними показниками конкурсних балів, що гарантує безперебійну роботу програми навіть при її першому запуску на новому комп'ютері.

2. ПРОЄКТУВАННЯ ПРОГРАМИ

2.1 Архітектура програми

Програмне забезпечення «Довідник абітурієнта» спроектовано та розроблено як класичний настільний застосунок Desktop Application з графічним інтерфейсом користувача GUI. Такий архітектурний підхід обрано з огляду на специфіку завдання: програма призначена для швидкої, автономної та локальної роботи користувача з масивами довідкової інформації без необхідності постійного підключення до мережі Інтернет чи використання віддалених серверів баз даних.

Для реалізації проекту обрано мову програмування C# та сучасну платформу .NET 8.0. Візуальна частина побудована з використанням технології Windows Forms, яка забезпечує швидку розробку інтуїтивно зрозумілого інтерфейсу та тісну інтеграцію з операційною системою Windows.

Архітектурно головне вікно програми поділено на дві ключові функціональні вкладки: «Довідник ВНЗ» та «Пошук». Вкладка «Довідник ВНЗ» відповідає за адміністрування даних: перегляд, додавання, редагування, видалення університетів та повне очищення бази. Вкладка «Пошук» інкапсулює аналітичну логіку: тут реалізовано алгоритми багатофакторної фільтрації, сортування спеціальностей за мінімальним конкурсним балом та швидке оновлення вартості навчання.

У програмі застосовано парадигму об'єктно-орієнтованого програмування (ООП). Усі сутності предметної області абстраговані у вигляді відповідних класів. Клас University моделює заклад вищої освіти, клас Specialty описує конкретну освітню програму, а клас AppManager виконує роль контролера, що управляє колекціями даних та файловими операціями.

Основна логіка взаємодії з користувачем, обробка подій, розміщена у класах вікон: головній формі та додаткових модальних діалогових вікнах - для додавання ЗВО та зміни ціни. Ці класи відповідають за зчитування введених даних, їхню первинну валідацію: перевірку на порожнечу або числовий формат, оновлення таблиць DataGridView та виклик відповідних методів бізнес-логіки.

Дані під час роботи програми зберігаються в оперативній пам'яті у вигляді типізованих колекцій List<T>. Зокрема, використовується список List<University>, де кожен об'єкт університету містить власну вкладену колекцію List<Specialty>. Для забезпечення постійного зберігання інформації реалізовано механізм запису стану цих колекцій у локальний текстовий файл. Завдяки

автоматичному фоновому збереженню, після закриття програми всі внесені користувачем зміни не втрачаються і безперешкодно завантажуються під час наступного запуску.

2.2 UML-діаграма варіантів використання

Для формалізованого опису функціональних вимог та моделювання взаємодії користувача з програмою було побудовано UML-діаграму варіантів використання Use Case Diagram. Ця діаграма наочно демонструє межі системи та основні дії - прецеденти, які доступні актору.

Основним і єдиним актором Actor у системі є «Користувач» - абітурієнт або адміністратор довідника. Він взаємодіє з графічним інтерфейсом настільного додатка для вирішення своїх завдань з пошуку та систематизації інформації.

До основних варіантів використання Use Cases, які підтримує розроблена система, належать:

- 1) перегляд бази університетів та спеціальностей;
- 2) багатофакторний пошук (за назвою, містом, формою навчання, фінансовим лімітом);
- 3) визначення мінімального конкурсного бала;
- 4) керування реєстром ЗВО (додавання, редагування, видалення);
- 5) актуалізація вартості контракту;
- 6) повне очищення бази даних;
- 7) автоматичне збереження та завантаження локального файлу.

UML-діаграму варіантів використання програми наведено на рисунку 2.1.



Рисунок 2.1 – UML-діаграма варіантів використання

2.3 UML-діаграма класів

Для відображення статичної структури програмного продукту, архітектури об'єктів та зв'язків між ними побудовано UML-діаграму класів Class Diagram. Вона деталізує типи даних, властивості, методи та рівні доступу для кожної логічної одиниці коду.

У програмі «Довідник абітурієнта» спроектовано та реалізовано наступні основні класи:

- 1) University – клас-контейнер, що описує заклад вищої освіти;
- 2) Specialty – клас, що містить детальні атрибути освітньої програми (прохідні бали, вартість);
- 3) AppManager – клас-контролер для управління колекціями та файловою підсистемою;
- 4) MainForm та допоміжні AddUniversityForm, UpdateFeeForm – класи графічного інтерфейсу, що успадковуються від базового класу Form фреймворку Windows Forms.

Ключовим архітектурним рішенням є використання відношення композиції між класами University та Specialty. Оскільки спеціальність не може існувати фізично та логічно без прив'язки до конкретного навчального закладу, клас університету жорстко інкапсулює у собі колекцію своїх спеціальностей.

UML-діаграму класів програми наведено на рисунку 2.2.

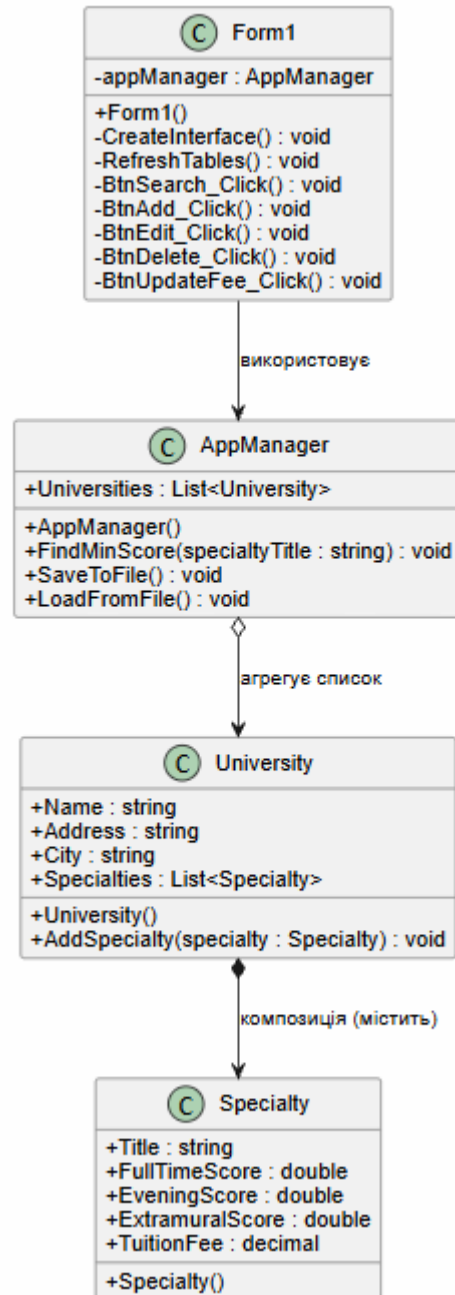


Рисунок 2.2 – UML-діаграма класів програми

На діаграмі графічно відображено, що головна форма програми агрегує у собі екземпляр класу **AppManager**, який, у свою чергу, керує глобальним списком **List<University>**. Такий розподіл відповідальності дозволяє уникнути надмірного навантаження на класи інтерфейсу, тобто самої форми, та робить код легко масштабованим і придатним для подальшої підтримки.

2.4 Структура класів програми

У програмі «Довідник абітурієнта» використано декілька основних класів, кожен з яких відповідає за окрему частину предметної області або за загальну роботу програми. Основними класами є University, Specialty, AppManager та класи графічного інтерфейсу.

2.4.1 Клас University

Клас University призначений для абстрагування та зберігання інформації про окремих заклад вищої освіти. Один екземпляр цього класу відповідає одному рядку у головній таблиці довідника. Клас містить такі основні властивості:

- 1) Name (тип string) – офіційна назва університету;
- 2) Address (тип string) – фактична адреса закладу;
- 3) City (тип string) – місто розташування;
- 4) Specialties (тип List<Specialty>) – колекція спеціальностей, що викладаються у цьому ЗВО.

Властивість Specialties є ключовою, оскільки вона реалізує відношення композиції. Завдяки цьому при видаленні об'єкта університету автоматично видаляються і всі пов'язані з ним спеціальності. Клас має конструктор для швидкої ініціалізації об'єкта базовими даними та метод AddSpecialty() для зручного розширення колекції освітніх програм.

2.4.2 Клас Specialty

Клас Specialty призначений для зберігання детальної інформації про конкретну освітню програму та умови вступу на неї. Клас містить такі властивості:

- 1) Title (тип string) – назва спеціальності (наприклад, «Інженерія програмного забезпечення»);
- 2) FullTimeScore (тип double) – прохідний конкурсний бал на денну форму навчання;
- 3) EveningScore (тип double) – прохідний конкурсний бал на вечірню форму навчання;
- 4) ExtramuralScore (тип double) – прохідний конкурсний бал на заочну форму;
- 5) TuitionFee (тип decimal) – вартість одного року навчання за контрактом.

Для відображення відсутності набору на певну форму навчання числові значення балів можуть приймати значення «0». У такому разі графічний

інтерфейс обробляє цей стан і виводить користувачу прочерк. Тип `decimal` для вартості обрано з метою забезпечення максимальної точності при роботі з фінансовими даними та фільтрації за ціною.

2.4.3 Клас `AppManager`

Клас `AppManager` виконує роль головного менеджера даних. Він відокремлює бізнес-логіку програми від візуального інтерфейсу, що відповідає принципам єдиної відповідальності `Single Responsibility Principle`. Основною властивістю класу є `Universities` – глобальний список `List<University>`, що містить усі поточні дані довідника. Клас містить такі ключові методи:

- 1) `SaveToFile()` – метод, що ініціює перетворення об'єктів у текстовий формат та їхній запис у локальний файл;
- 2) `LoadFromFile()` – метод, що зчитує файл, відновлює об'єкти та заповнює колекцію `Universities`;
- 3) Службові методи генерації еталонної бази даних у разі першого запуску.

2.4.4 Класи графічного інтерфейсу

Графічна підсистема програми реалізована за допомогою класів, успадкованих від `System.Windows.Forms.Form`. Головна форма програми `MainDashboard` відповідає за створення базового вікна, вкладок, таблиць `DataGridView` та обробку подій натискання кнопок. У цьому класі зберігається єдиний екземпляр `AppManager`. Метод `RefreshTables()` використовується для синхронізації візуальних таблиць із даними в оперативній пам'яті. Методи пошуку та фільтрації, реалізовані у подіях кнопок, проходять циклами по колекціях `AppManager` і формують відсортований список результатів. Додаткові класи `AddUniversityForm` та `UpdateFeeForm` відповідають за відображення модальних діалогових вікон для введення даних користувачем (додавання ЗВО та оновлення вартості відповідно). Вони містять логіку валідації текстових полів.

2.5 Спосіб зберігання даних

Для забезпечення збереження даних між частинами роботи у програмі «Довідник абітурієнта» використовується локальний текстовий файл. Цей підхід обрано як найбільш оптимальний для настільного додатка-довідника, оскільки він не вимагає встановлення сторонніх систем управління базами даних СУБД та легко переноситься між комп'ютерами.

Збереження виконується повністю в автоматичному режимі. Після кожної дії користувача, яка призводить до зміни стану даних (додавання, редагування,

видалення об'єктів), метод `SaveToFile()` перезаписує файл актуальною інформацією. Програма автоматично перетворює складні об'єкти з оперативної пам'яті у звичайний текстовий формат, JSON або структурований текст, і записує їх у файл. При цьому зберігається вся правильна ієрархія: кожен університет чітко містить свій власний список спеціальностей та цін.

Файл збереження створюється безпосередньо у робочій директорії програми або у папці документів користувача. Під час запуску додатка метод `LoadFromFile()` перевіряє наявність цього файлу. Якщо файл існує і не пошкоджений, програма зчитує з нього текст, автоматично відновлює на його основі повноцінні об'єкти C# та завантажує їх в оперативну пам'ять. Якщо файл відсутній, наприклад при першому запуску програми, система самостійно генерує демонстраційний набір даних із 13 провідних університетів України та одразу створює новий файл збереження.

2.6 Інструкція зі встановлення та запуску програми

Вихідний код програми «Довідник абітурієнта» розміщено у репозиторії GitHub за посиланням:

<https://github.com/yelyzavetaholovko-ui/Applicant-directory>

Для запуску програми з репозиторію необхідно мати таке програмне забезпечення:

- 1) операційну систему Windows;
- 2) середовище розробки Microsoft Visual Studio;
- 3) встановлену платформу .NET;
- 4) доступ до мережі Інтернет для завантаження проєкту з GitHub.

Порядок встановлення та запуску програми:

- 1) відкрити сторінку репозиторію GitHub з вихідним кодом програми;
- 2) натиснути кнопку «Code»;
- 3) обрати пункт «Download ZIP»;
- 4) після завершення завантаження розпакувати ZIP-архів у зручну папку на комп'ютері;
- 5) відкрити розпаковану папку з проєктом;
- 6) запустити файл проєкту `ApplicantDirectory.csproj` (або файл рішення `ApplicantDirectory.sln`) у середовищі Microsoft Visual Studio;
- 7) дочекатися завантаження проєкту та відновлення необхідних залежностей;
- 8) переконатися, що вибрано конфігурацію `Debug` або `Release`;

- 9) натиснути кнопку запуску програми або клавішу F5;
- 10) після запуску відкриється головне вікно програми «Довідник абітурієнта».

Після першого запуску програма перевіряє наявність локального файлу збереження даних. Якщо файл відсутній, програма автоматично генерує початковий набір демонстраційних даних із 13 провідних університетів України. Якщо файл уже існує, програма завантажує з нього раніше збережену інформацію.

Для повторного запуску програми достатньо знову відкрити проєкт у Microsoft Visual Studio та натиснути F5. Дані, які були додані або змінені користувачем під час попередньої роботи, зберігаються автоматично.

Для видалення програми потрібно просто видалити папку з розпакованим проєктом. Якщо потрібно повністю видалити також усі збережені дані, необхідно додатково видалити згенерований файл бази даних із папки програми або папки документів користувача.

ВИСНОВКИ

У результаті виконання курсової роботи було розроблено програму «Довідник абітурієнта», призначену для зберігання, пошуку та обробки інформації про заклади вищої освіти (ЗВО) та їхні освітні програми. Програма дозволяє користувачу вести список університетів, додавати, редагувати та видаляти записи, здійснювати пошук спеціальностей, а також фільтрувати результати за формою навчання, містом та вартістю контракту.

Під час розробки було реалізовано декілька основних сутностей предметної області: університет `University` та спеціальність `Specialty`. Завдяки використанню відношення композиції у програмі реалізовано ієрархічну структуру даних, за якої кожен університет містить власну колекцію спеціальностей, що автоматично видаляються або оновлюються разом із основним закладом. Управління всіма колекціями інкапсульовано у класі `AppManager`.

У програмі реалізовано графічний інтерфейс користувача з базовим вікном `MainDashboard`, яке поділено на зручні вкладки «Пошук» та «Довідник ВНЗ». Для введення нових даних та зміни існуючих розроблено додаткові модальні вікна `AddUniversityForm`, `UpdateFeeForm`. Така структура інтерфейсу дозволяє зручно розділити основні функції програми, уникнути перевантаження головного екрана та спростити роботу користувача з даними.

Для збереження інформації між запусками програми використовується локальний текстовий файл. У ньому зберігаються списки університетів та вся вкладена інформація про спеціальності, ціни і прохідні бали. Завдяки автоматичному збереженню та завантаженню користувач не втрачає введені дані після закриття програми, а у разі першого запуску система самостійно формує початкову базу даних.

Розроблена програма відповідає поставленій меті курсової роботи. Вона реалізує основні операції з колекціями даних: додавання, редагування, видалення, пошук, фільтрацію, збереження та завантаження інформації. Також у програмі передбачено специфічні функції для абітурієнта: обчислення мінімального прохідного бала, відбір закладів за фінансовим лімітом та швидке оновлення вартості навчання.

У подальшому програму можна вдосконалити, додавши розширену валідацію введених текстових даних, можливість експорту результатів пошуку у текстовий або PDF-файл для друку, інтеграцію калькулятора конкурсного бала на основі результатів НМТ, а також можливість використання повноцінної бази

даних замість звичайного текстового файлу. Це дозволило б зробити програму більш зручною для практичного використання та розширити її функціональні можливості.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1.Бондарєв В. М. Конспект лекцій з дисципліни «Об’єктно-орієнтоване програмування».

<https://tss.co.ua:7074/>

2.Common C# code conventions. Microsoft Learn.

<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>

3.Windows Forms documentation. Microsoft Learn.

<https://learn.microsoft.com/en-us/dotnet/desktop/winforms/>

4.System.Text.Json documentation. Microsoft Learn.

<https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview>

5.PlantUML Documentation.

<https://plantuml.com/>

6.ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. Київ : ДП «УкрНДНЦ», 2016. 26 с.

https://software.nure.ua/wp-content/uploads/2021/02/DSTU_3008-2015.pdf

7.ДСТУ 8302:2015. Інформація та документація. Бібліографічне посилання. Загальні положення та правила складання. Київ : ДП «УкрНДНЦ», 2016. 16 с.

<https://software.nure.ua/wp-content/uploads/2021/02/dstu-8302-2015.pdf>